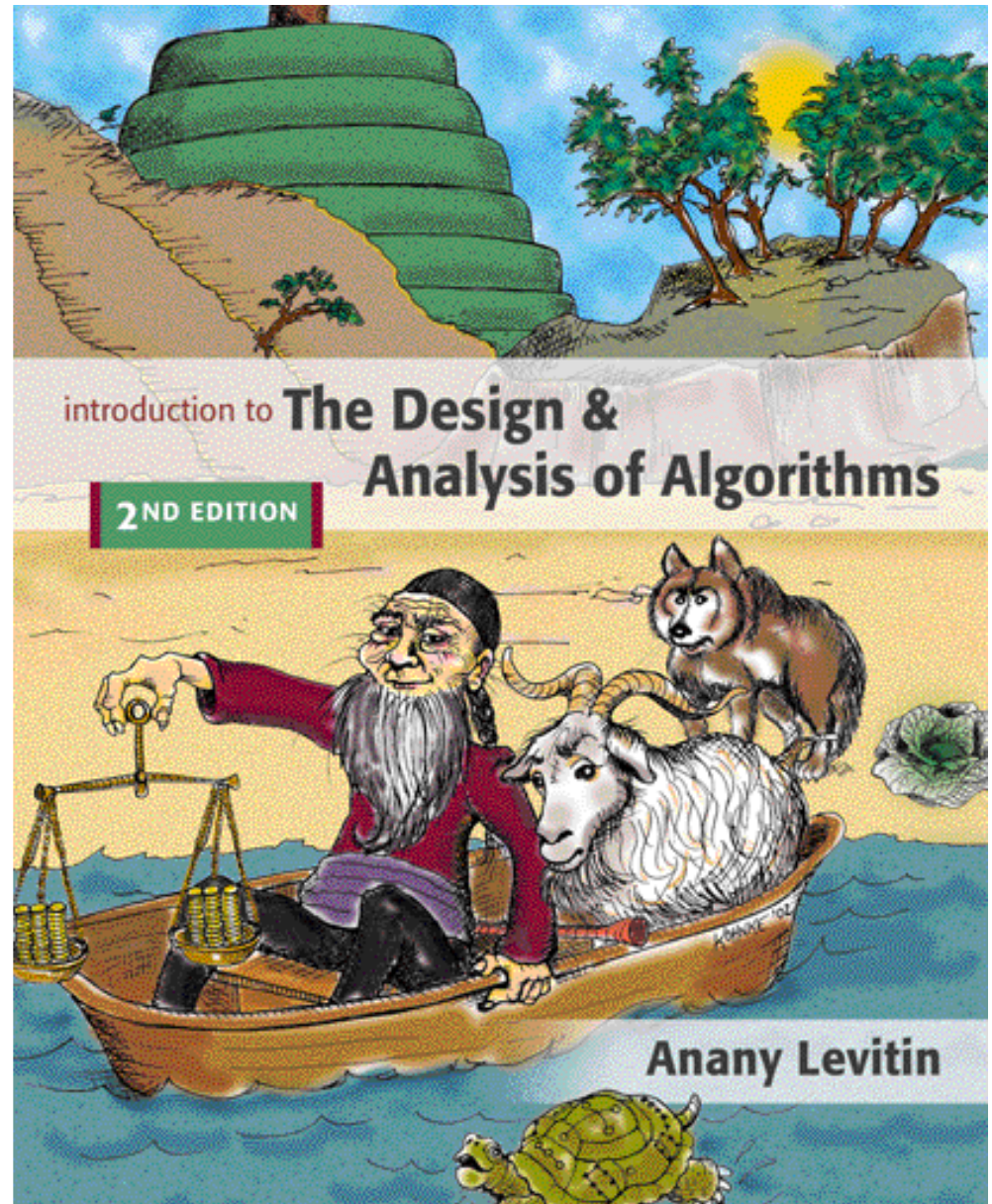
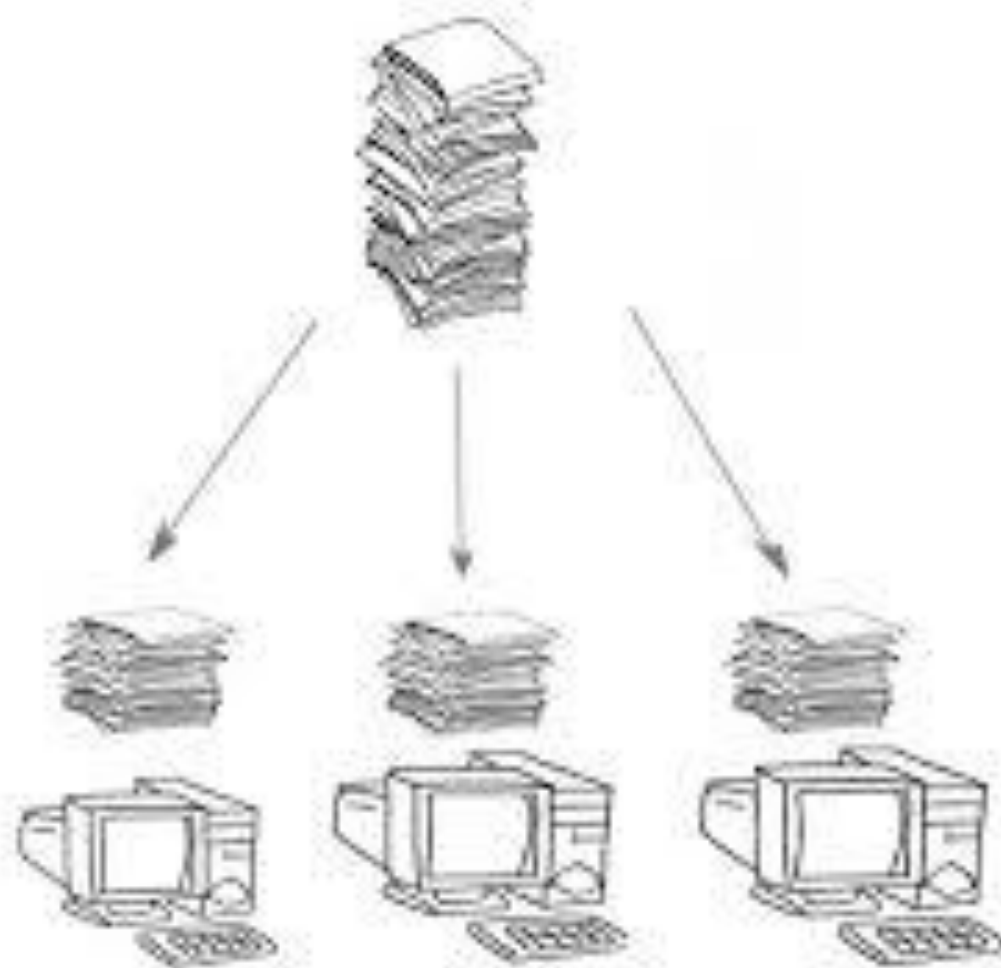


Chapter 5

Divide-and-Conquer



Divide and Conquer

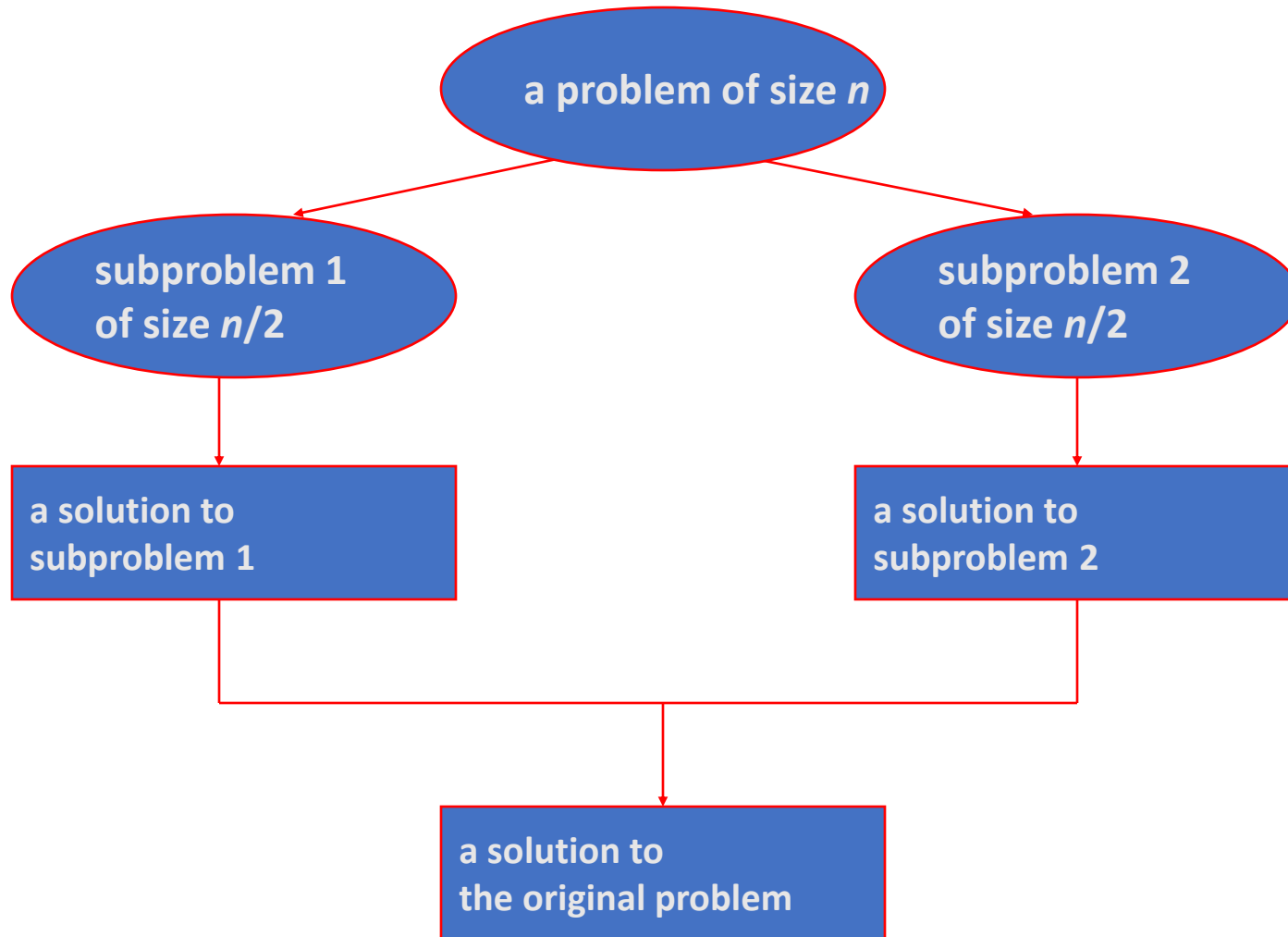


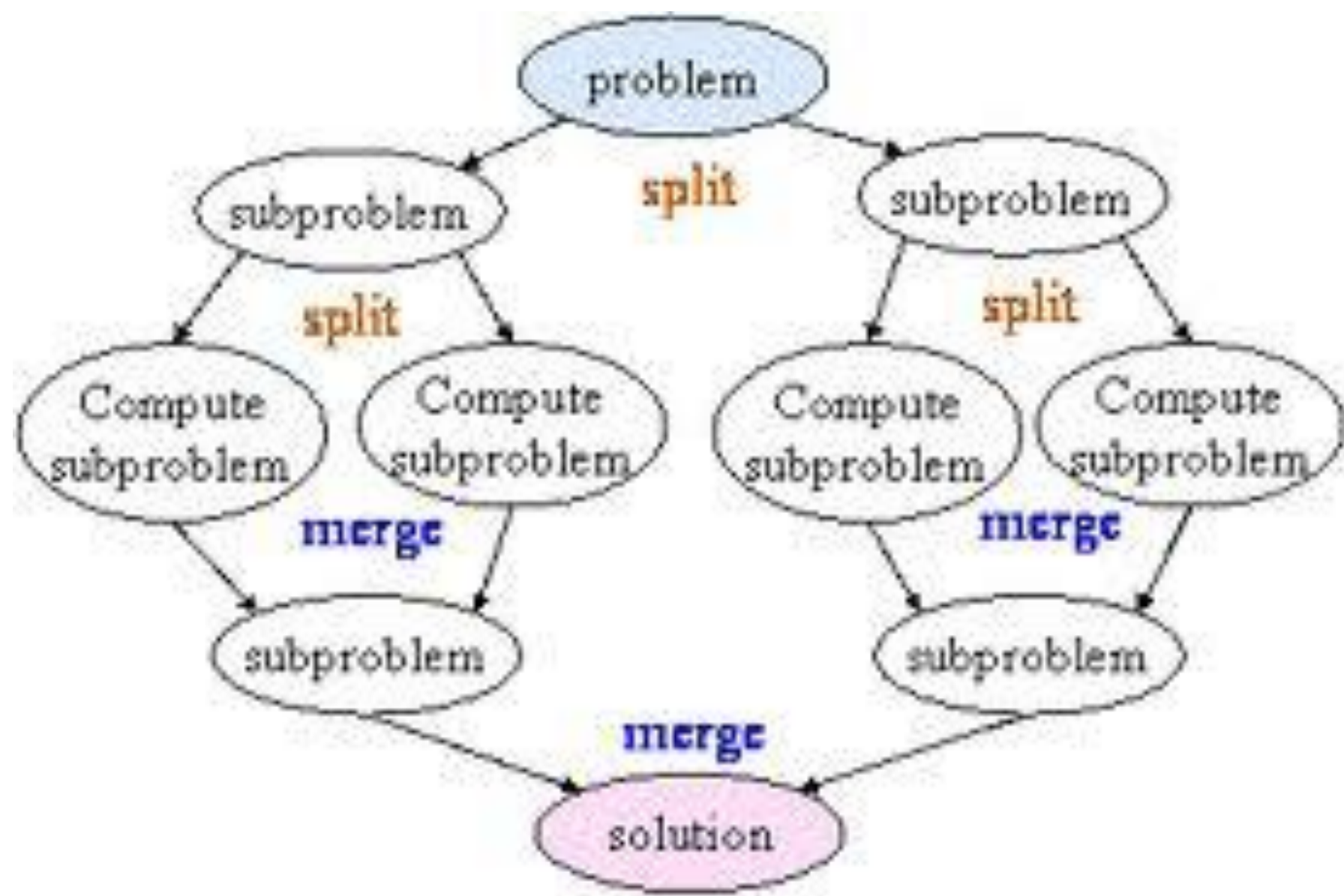
Divide-and-Conquer

The most-well known algorithm design strategy:

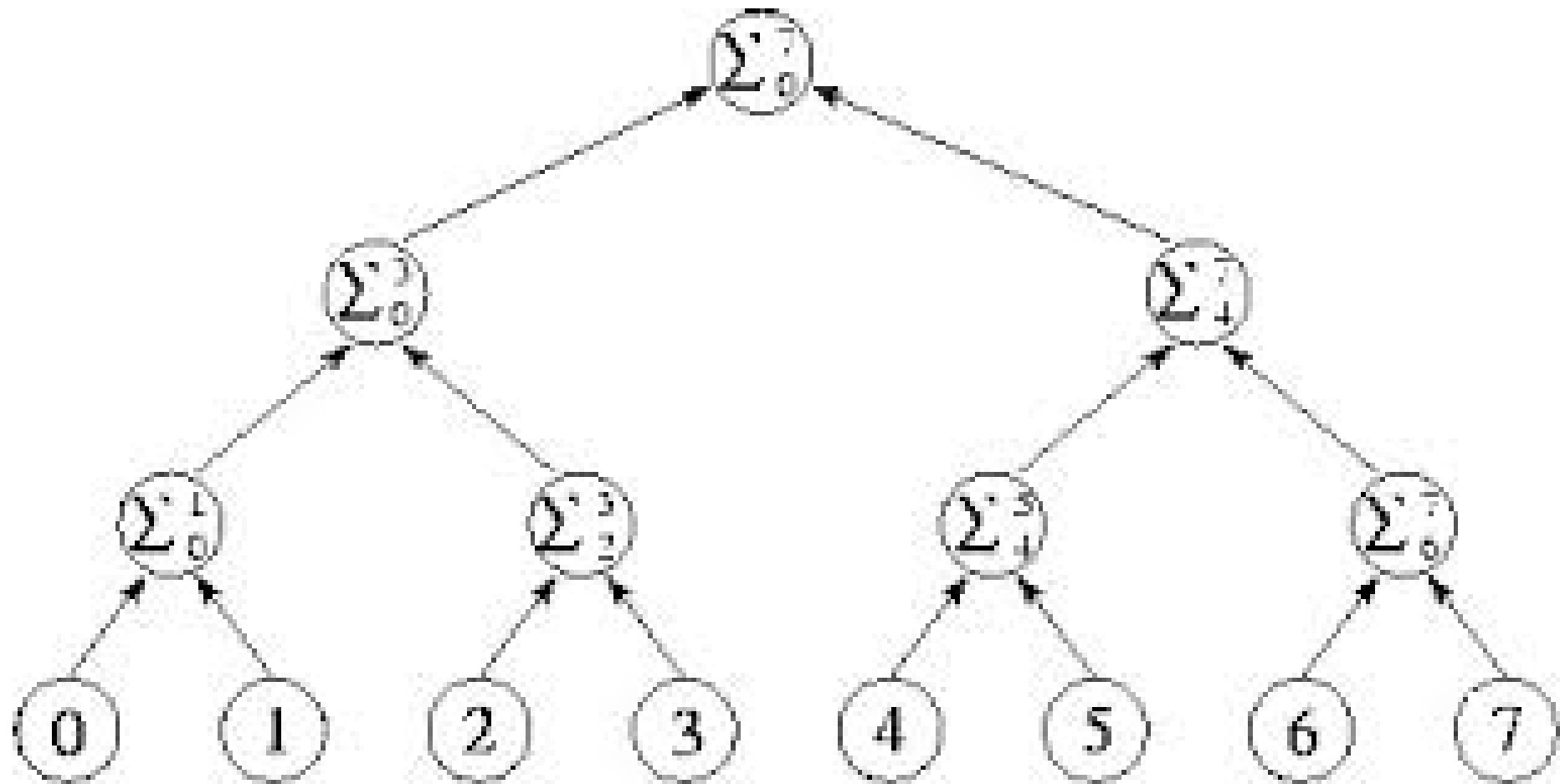
1. Divide instance of problem into two or more smaller instances
2. Solve smaller instances recursively
3. Obtain solution to original (larger) instance by combining these solutions

Divide-and-Conquer Technique (cont.)





Summation Σ



General Divide-and-Conquer Recurrence

Summation:

$$a_0 + \dots a_{n-1} = (a_0 + \dots a_{\lfloor n/2 \rfloor - 1}) + (a_{\lfloor n/2 \rfloor} + \dots a_{n-1})$$

if $n = 1$, we simply return a_0 as the answer.

Divide-and-Conquer Examples

- Sorting: mergesort and quicksort
 - Binary tree traversals
 - Multiplication of large integers
 - Matrix multiplication: Strassen's algorithm
 - Closest-pair and convex-hull algorithms
 - Binary search: decrease-by-half (or degenerate divide&conq.)
-

General Divide-and-Conquer Recurrence

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) \in \Theta(n^d), \quad d \geq 0$$

Input size n is divided into b instances of size n/b while a of them need to be solved.

$f(n)$ is a function accounting for time spent on dividing the problem into a smaller one and combining their solutions.

General Divide-and-Conquer Recurrence

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) \in \Theta(n^d), \quad d \geq 0$$

Master Theorem: If $a < b^d$, $T(n) \in \Theta(n^d)$

$$\text{If } a = b^d, \quad T(n) \in \Theta(n^d \log n)$$

$$\text{If } a > b^d, \quad T(n) \in \Theta(n^{\log_b a})$$

Note: The same results hold with O instead of Θ .

$$\text{Examples: } T(n) = 4T(n/2) + n \Rightarrow T(n) \in ?$$

$$T(n) = 4T(n/2) + n^2 \Rightarrow T(n) \in ?$$

$$T(n) = 4T(n/2) + n^3 \Rightarrow T(n) \in ?$$

General Divide-and-Conquer Recurrence

For summation of input size $n=2^k$:

$$A(n) = 2A(n/2) + 1$$

$a=2$, $b=2$ and $d=0$; hence since $a > b^d$

$$A(n) \in \Theta(n^{\log_2 2}) = \Theta(n)$$

Master Theorem: If $a < b^d$, $T(n) \in \Theta(n^d)$
 If $a = b^d$, $T(n) \in \Theta(n^d \log n)$
 If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

Master Theorem: If $a < b^d$, $T(n) \in \Theta(n^d)$
 If $a = b^d$, $T(n) \in \Theta(n^d \log n)$
 If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

Master Theorem: If $a < b^d$, $T(n) \in \Theta(n^d)$
 If $a = b^d$, $T(n) \in \Theta(n^d \log n)$
 If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

Mergesort

- Split array $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows:
 - Repeat the following until no elements remain in one of the arrays:
 - compare the first elements in the remaining unprocessed portions of the arrays
 - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
 - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Pseudocode of Mergesort

ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n - 1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A)

Pseudocode of Merge

ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted

//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C

$i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow 0$

while $i < p$ **and** $j < q$ **do**

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$; $i \leftarrow i + 1$

else $A[k] \leftarrow C[j]$; $j \leftarrow j + 1$

$k \leftarrow k + 1$

if $i = p$

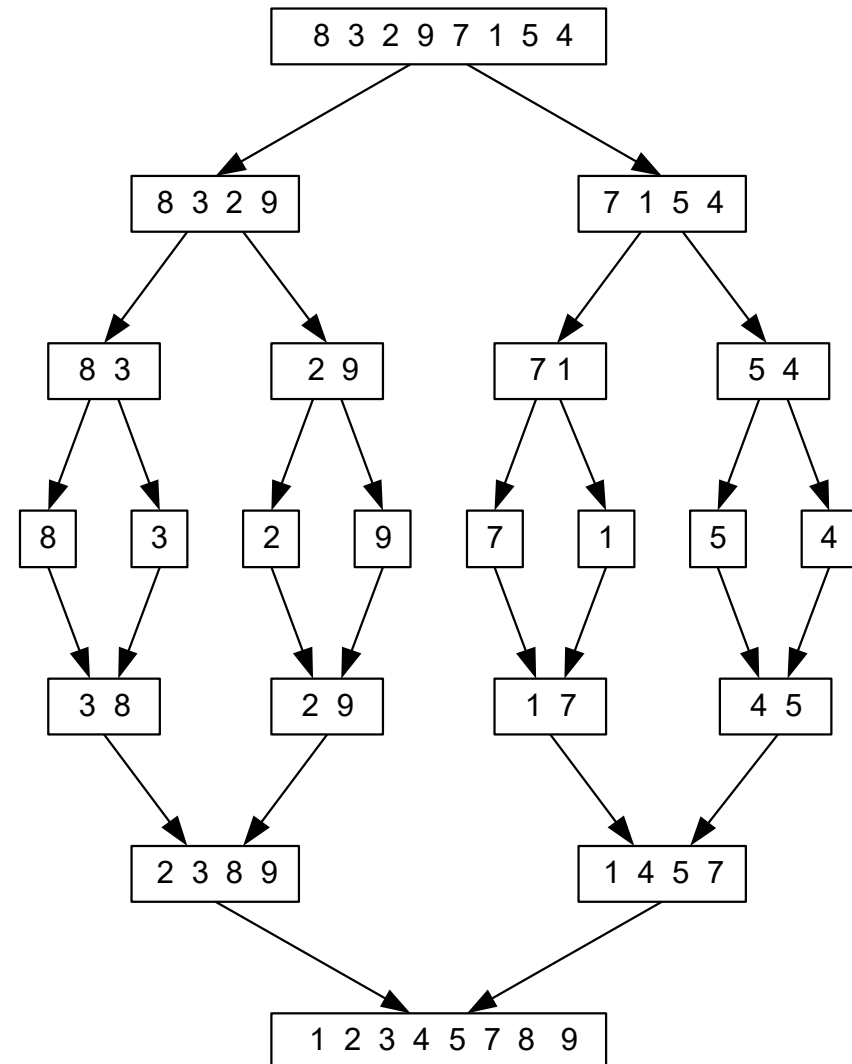
 copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

Mergesort Example

```

 $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$ 
while  $i < p$  and  $j < q$  do
    if  $B[i] \leq C[j]$ 
         $A[k] \leftarrow B[i]; i \leftarrow i + 1$ 
    else  $A[k] \leftarrow C[j]; j \leftarrow j + 1$ 
     $k \leftarrow k + 1$ 
if  $i = p$ 
    copy  $C[j..q - 1]$  to  $A[k..p + q - 1]$ 
else copy  $B[i..p - 1]$  to  $A[k..p + q - 1]$ 
    
```



Analysis of Mergesort

$$C(n) = 2 C(n/2) + C_{\text{merge}}(n) \text{ for } n > 1 \text{ and } C(1) = 0$$

$$C_{\text{merge}}(n) = n - 1$$

$$C_{\text{worst}}(n) = 2 C_{\text{worst}}(n/2) + n - 1$$

$$C_{\text{worst}}(n) \in \Theta(n \log n)$$

Analysis of Mergesort

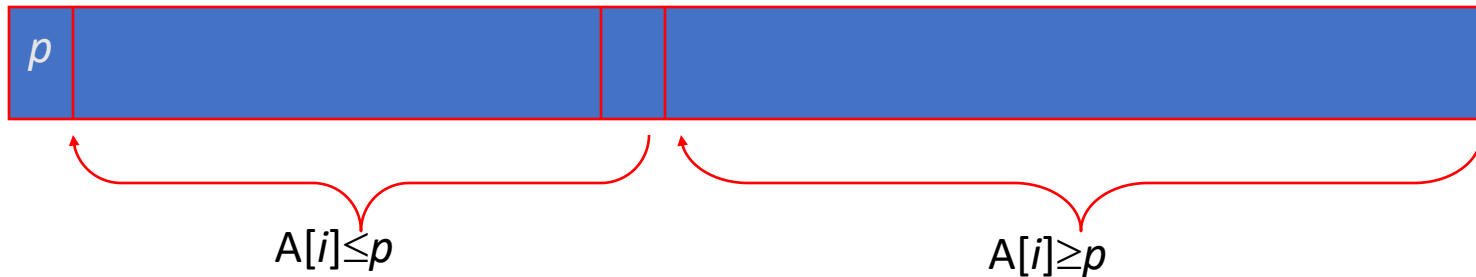
- All cases have same efficiency: $\Theta(n \log n)$
- Number of comparisons in the worst case is close to theoretical minimum for comparison-based sorting:

$$\lceil \log_2 n! \rceil \approx n \log_2 n - 1.44n$$

- Space requirement: $\Theta(n)$ (not in-place)
- Can be implemented without recursion (bottom-up)

Quicksort

- Select a *pivot* (partitioning element) – here, the first element
- Rearrange the list so that all the elements in the first s positions are smaller than or equal to the pivot and all the elements in the remaining $n-s$ positions are larger than or equal to the pivot (see next slide for an algorithm)



- Exchange the pivot with the last element in the first (i.e., $\leq$) subarray — the pivot is now in its final position
- Sort the two subarrays recursively

Hoare's Partitioning Algorithm

Algorithm *Partition*($A[l..r]$)

//Partitions a subarray by using its first element as a pivot

//Input: A subarray $A[l..r]$ of $A[0..n - 1]$, defined by its left and right

// indices l and r ($l < r$)

//Output: A partition of $A[l..r]$, with the split position returned as

// this function's value

$p \leftarrow A[l]$

$i \leftarrow l; \quad j \leftarrow r + 1$

repeat

repeat $i \leftarrow i + 1$ **until** $A[i] \geq p$

repeat $j \leftarrow j - 1$ **until** $A[j] \leq p$

 swap($A[i], A[j]$)

until $i \geq j$

swap($A[i], A[j]$) //undo last swap when $i \geq j$

swap($A[l], A[j]$)

return j

Quicksort Example

5 3 1 9 8 2 4 7

Analysis of Quicksort

- Best case: split in the middle

The total number of key comparison in partition;

$A[i] \geq p$ & $A[j] \leq p$, is $n + 1$

$$C_{\text{best}}(n) = 2C_{\text{best}}(n/2) + n \quad \text{for } n > 1, C_{\text{best}}(1) = 0$$

According to Master Theorem,

$$C_{\text{best}}(n) = \Theta(n \log n)$$

Analysis of Quicksort

- Worst case: sorted array!

The total number of key comparison made will be equal to

$$\begin{aligned}C_{\text{worst}}(n) &= (n+1) + n + \dots + 3 \\&= (n+1)(n+2) / 2 - 3 \\&= \Theta(n^2)\end{aligned}$$

Analysis of Quicksort

- Average case: random arrays — $\Theta(n \log n)$

Assuming that partition split can happen in each position s ($0 \leq s \leq n-1$) with the same probability $1/n$.

$$C_{\text{avg}}(n) = \frac{1}{n} \sum_{s=0}^{n-1} [(n+1) + C_{\text{avg}}(s) + C_{\text{avg}}(n-1-s)] \quad ; n > 1$$

$$C_{\text{avg}}(0) = 0, C_{\text{avg}}(1) = 0$$

$$C_{\text{avg}}(n) \approx 2n \ln(n) \approx 1.38n \lg(n)$$

Analysis of Quicksort

- Best case: split in the middle — $\Theta(n \log n)$
- Worst case: sorted array! — $\Theta(n^2)$
- Average case: random arrays — $\Theta(n \log n)$
- Improvements:
 - better pivot selection: median of three partitioning
 - switch to insertion sort on small subfiles
 - elimination of recursionThese combine to 20-25% improvement
- Considered the method of choice for internal sorting of large files ($n \geq 10000$)

Binary Search

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
value	5	20	32	35	45	67	68	72	75	77	81	82	97
1 st	l						m						r
2 nd								l		m			r
3 rd								lm		r			

- What is the recursive algorithm for a binary search?

Binary Search

recBinS(A[0..n-1,K)



Binary Search

Very efficient algorithm for searching in sorted array:

K

vs

$A[0] \dots A[m] \dots A[n-1]$

If $K = A[m]$, stop (successful search); otherwise, continue searching by the same method in $A[0..m-1]$ if $K < A[m]$ and in $A[m+1..n-1]$ if $K > A[m]$

$l \leftarrow 0; \quad r \leftarrow n-1$

while $l \leq r$ do

$m \leftarrow \lfloor (l+r)/2 \rfloor$

 if $K = A[m]$ return m

 else if $K < A[m]$ $r \leftarrow m-1$

 else $l \leftarrow m+1$

return -1

Analysis of Binary Search

- Time efficiency
 - worst-case recurrence: $C_w(n) = 1 + C_w(\lfloor n/2 \rfloor)$, $C_w(1) = 1$
solution: $C_w(n) = \lceil \log_2(n+1) \rceil$

This is VERY fast: e.g., $C_w(10^6) = 20$

- Optimal for searching a sorted array
- Limitations: must be a sorted array (not linked list)
- Bad (degenerate) example of divide-and-conquer
- Has a continuous counterpart called *bisection method* for solving equations in one unknown $f(x) = 0$ (see Sec. 12.4)

Binary Tree Algorithms

Binary tree is a divide-and-conquer ready structure!

Ex. 1: Classic traversals (preorder, inorder, postorder)

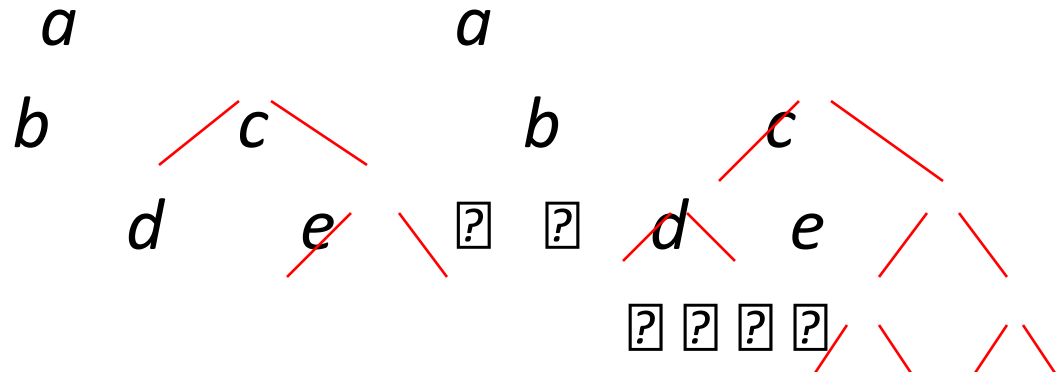
Algorithm *Inorder*(T)

if $T \neq \emptyset$

Inorder(T_{left})

print(root of T)

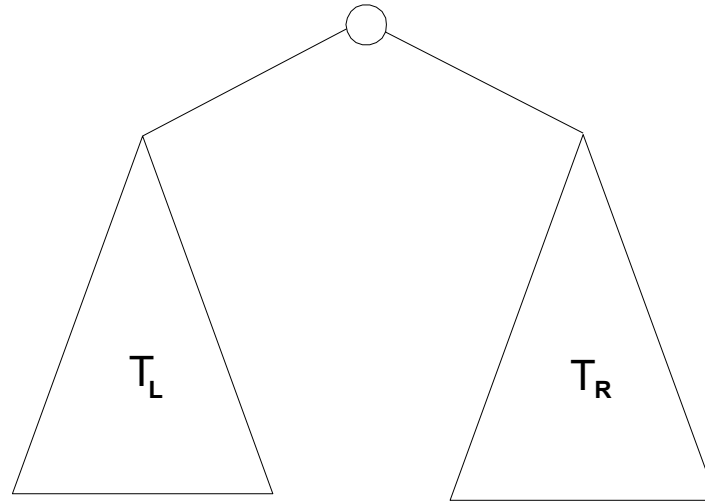
Inorder(T_{right})



Efficiency: $\Theta(n)$

Binary Tree Algorithms (cont.)

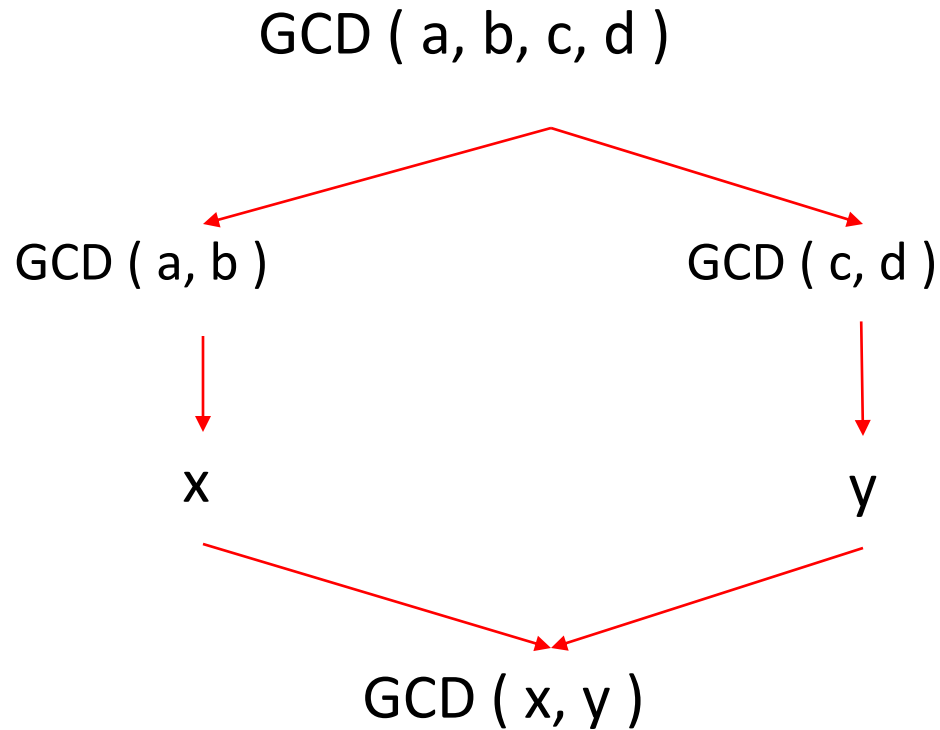
Ex. 2: Computing the height of a binary tree



$$h(T) = \max\{h(T_L), h(T_R)\} + 1 \text{ if } T \neq \emptyset \text{ and } h(\emptyset) = -1$$

Efficiency: $\Theta(n)$

$\text{GCD} (a, b, c, d)$



GCD (a, b, c)

$$\text{GCD} (a, b, c) = \text{GCD} (a, \text{GCD} (b, c))$$

$$\text{GCD} (a, b, c) = \begin{cases} a & \text{if } b=0 \\ \text{GCD} (b, c, a \% b) & \text{otherwise} \end{cases}$$

Multiplication of Large Integers

Consider the problem of multiplying two (large) n -digit integers represented by arrays of their digits such as:

$A = 12345678901357986429$ $B = 87654321284820912836$

The grade-school algorithm:

$$\begin{array}{r} a_1 \ a_2 \ \dots \ a_n \\ b_1 \ b_2 \ \dots \ b_n \\ (d_{10}) \ d_{11} \ d_{12} \ \dots \ d_{1n} \\ \hline (d_{20}) \ d_{21} \ d_{22} \ \dots \ d_{2n} \\ \dots \ \dots \ \dots \ \dots \ \dots \ \dots \ \dots \\ (d_{n0}) \ d_{n1} \ d_{n2} \ \dots \ d_{nn} \end{array}$$

Efficiency: n^2 one-digit multiplications

Multiplication of Large Integers

$$23 = 2 \cdot 10^1 + 3 \cdot 10^0 \text{ and } 14 = 1 \cdot 10^1 + 4 \cdot 10^0$$

$$\begin{aligned} 23 * 14 &= (2 \cdot 10^1 + 3 \cdot 10^0) * (1 \cdot 10^1 + 4 \cdot 10^0) \\ &= (2 \cdot 1)10^2 + (2 \cdot 4 + 3 \cdot 1)10^1 + (3 \cdot 4)10^0 \end{aligned}$$

However

$$2 \cdot 4 + 3 \cdot 1 = (2+3) * (1+4) - 2 \cdot 1 - 3 \cdot 4$$

$$(a+b)(c+d)$$

$$a \cdot d + b \cdot c = (a+b)(c+d) - ac - bd?$$

Multiplication of Large Integers

$$a = a_1a_0 \quad \text{and} \quad b = b_1b_0$$

$$c = a * b = c_2 10^2 + c_1 10^1 + c_0$$

$$\text{where } c_2 = a_1 * b_1$$

$$c_0 = a_0 * b_0$$

$$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$$

Applying divide and conquer: for n-digit integers a and b.

Anatoly Karatsuba,
Russian Mathematician



First Divide-and-Conquer Algorithm

A small example: $A * B$ where $A = 2135$ and $B = 4014$

$$A = (21 \cdot 10^2 + 35), \quad B = (40 \cdot 10^2 + 14)$$

$$\begin{aligned} \text{So, } A * B &= (21 \cdot 10^2 + 35) * (40 \cdot 10^2 + 14) \\ &= 21 * 40 \cdot 10^4 + (21 * 14 + 35 * 40) \cdot 10^2 + 35 * 14 \end{aligned}$$

In general, if $A = A_1A_2$ and $B = B_1B_2$ (where A and B are n -digit, A_1, A_2, B_1, B_2 are $n/2$ -digit numbers),

$$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{n/2} + A_2 * B_2$$

Recurrence for the number of one-digit multiplications $M(n)$:

$$M(n) = 4M(n/2), \quad M(1) = 1$$

Solution: $M(n) = n^2$

Second Divide-and-Conquer Algorithm

$$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{n/2} + A_2 * B_2$$

The idea is to decrease the number of multiplications from 4 to 3:

$$(A_1 + A_2) * (B_1 + B_2) = A_1 * B_1 + (A_1 * B_2 + A_2 * B_1) + A_2 * B_2,$$

I.e., $(A_1 * B_2 + A_2 * B_1) = (A_1 + A_2) * (B_1 + B_2) - A_1 * B_1 - A_2 * B_2$,
which requires only 3 multiplications at the expense of $(4-1)$ extra add/sub.

Recurrence for the number of multiplications $M(n)$:

$$M(n) = 3M(n/2) + \Theta(n), \quad M(1) = 1$$

Solution: $M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}$

Example of Large-Integer Multiplication

2135 * 4014

Brute Force for Matrix Multiplication

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two n -by- n matrices by the definition-based algorithm

//Input: Two n -by- n matrices A and B

//Output: Matrix $C = AB$

for $i \leftarrow 0$ **to** $n - 1$ **do**

for $j \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow 0.0$

for $k \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

return C

Strassen's Matrix Multiplication

Strassen observed [1969] that the product of two matrices can be computed as follows:

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} * \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix}$$

$$= \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{pmatrix}$$

Formulas for Strassen's Algorithm

$$M_1 = (A_{00} + A_{11}) * (B_{00} + B_{11})$$

$$M_2 = (A_{10} + A_{11}) * B_{00}$$

$$M_3 = A_{00} * (B_{01} - B_{11})$$

$$M_4 = A_{11} * (B_{10} - B_{00})$$

$$M_5 = (A_{00} + A_{01}) * B_{11}$$

$$M_6 = (A_{10} - A_{00}) * (B_{00} + B_{01})$$

$$M_7 = (A_{01} - A_{11}) * (B_{10} + B_{11})$$

Analysis of Strassen's Algorithm

If n is not a power of 2, matrices can be padded with zeros.

Number of multiplications:

$$M(n) = 7M(n/2), \quad M(1) = 1$$

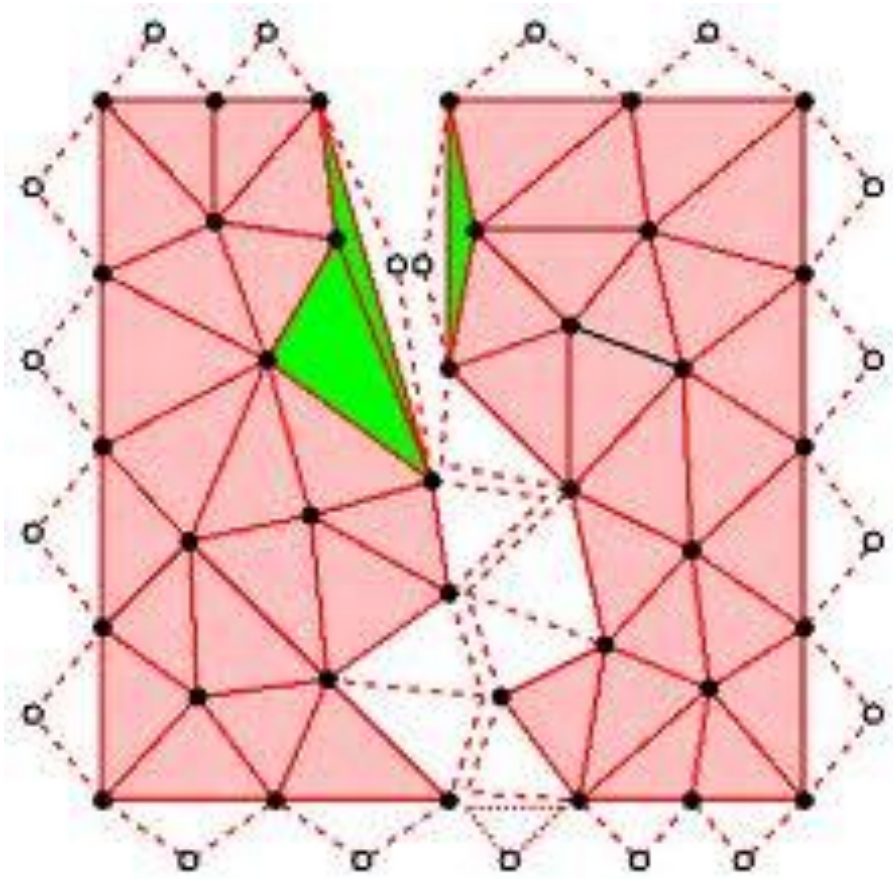
Solution: $M(n) = 7^{\log_2 n} = n^{\log_2 7} \approx n^{2.807}$ vs. n^3 of brute-force alg.

Algorithms with better asymptotic efficiency are known but they are even more complex.

Conquer the world



Conquer the geographic

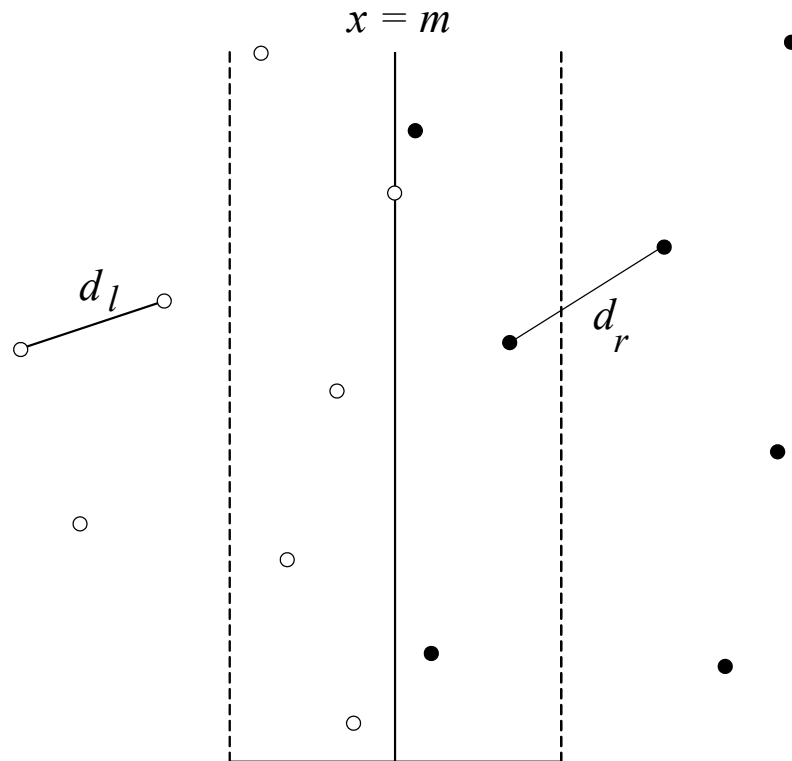


Is it easy to divide?



Closest-Pair Problem by Divide-and-Conquer

Step 1 Divide the points given into two subsets P_l and P_r by a vertical line $x = m$ so that half the points lie to the left or on the line and half the points lie to the right or on the line.



Closest Pair by Divide-and-Conquer (cont.)

Step 2 Find recursively the closest pairs for the left and right subsets.

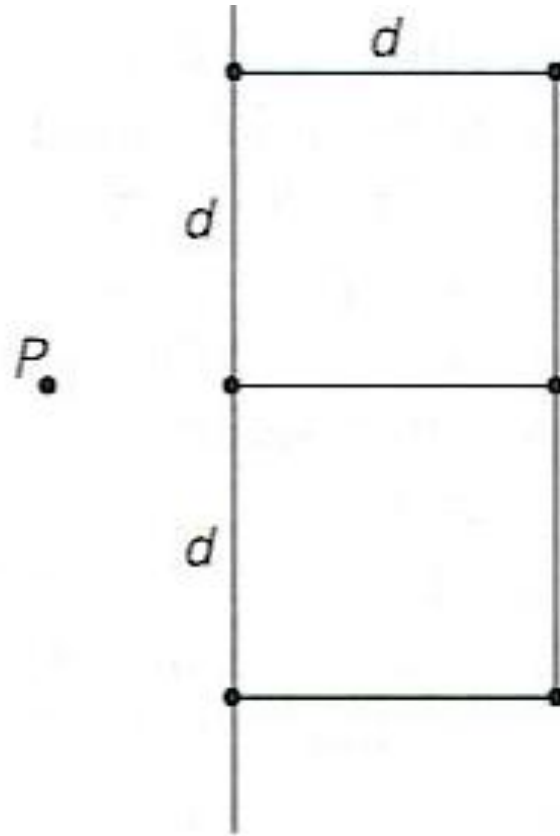
Step 3 Set $d = \min\{d_l, d_r\}$

We can limit our attention to the points in the symmetric vertical strip S of width $2d$ as possible closest pair. (The points are stored and processed in increasing order of their y coordinates.)

Step 4 Scan the points in the vertical strip S from the lowest up. For every point $p(x,y)$ in the strip, inspect points in the strip that may be closer to p than d . There can be no more than 5 such points following p on the strip list!

Closest Pair by Divide-and-Conquer: Worst Case

The worst case scenario is depicted below:



Efficiency of the Closest-Pair Algorithm

Running time of the algorithm is described by

$$T(n) = 2T(n/2) + M(n), \text{ where } M(n) \in O(n)$$

By the Master Theorem (with $a = 2$, $b = 2$, $d = 1$)

$$T(n) \in O(n \log n)$$

Algorithm

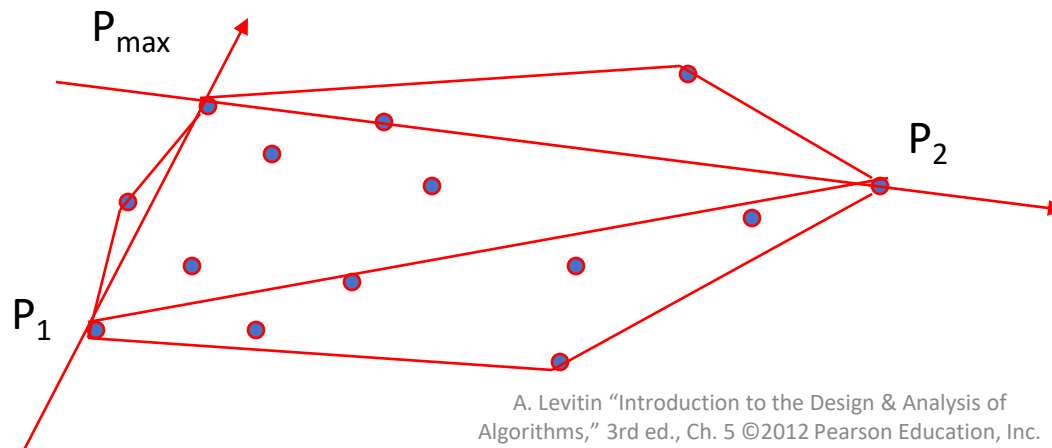
Algorithm

Algorithm

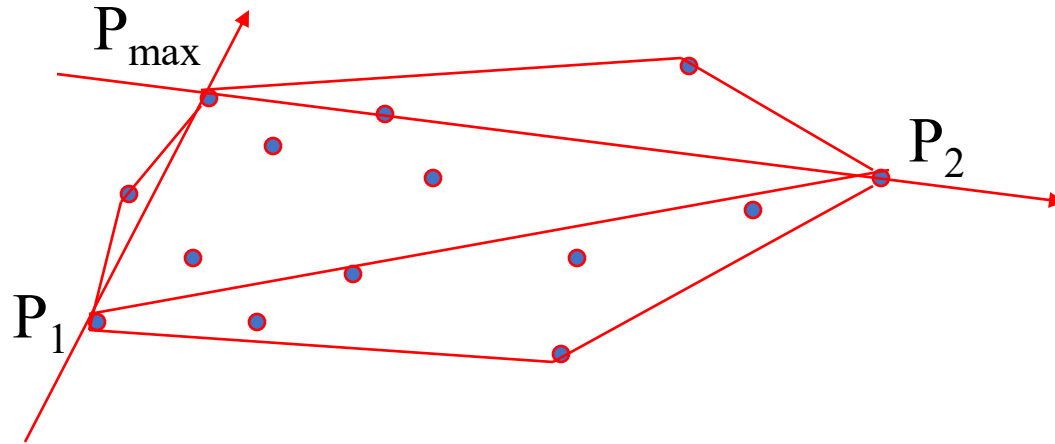
Quickhull Algorithm

Convex hull: smallest convex set that includes given points

- Assume points are sorted by x-coordinate values
- Identify *extreme points* P_1 and P_2 (leftmost and rightmost)
- Compute *upper hull* recursively:
 - find point $P_{\max}$ that is farthest away from line P_1P_2
 - compute the upper hull of the points to the left of line $P_1P_{\max}$
 - compute the upper hull of the points to the left of line $P_{\max}P_2$
- Compute *lower hull* in a similar manner



Quickhull Algorithm



- $P_{\max}$ is a vertex of the upper hull
- Points inside $\triangle P_1 P_{\max} P_2$ cannot be vertices of the upper hull.
- There are no points to the left of BOTH lines $P_1 P_{\max}$ and $P_{\max} P_2$.

Distance from Point to Line

- If $p_1=(x_1,y_1)$, $p_2=(x_2,y_2)$, $p_3(x_3,y_3)$ then area of $\Delta p_1p_2p_3$
 $= |x_1y_2 + x_2y_3 + x_3y_1 - x_1y_3 - x_2y_1 - x_3y_2|/2$
- Area of $\Delta p_1p_2p_3$ is, also, $= \frac{1}{2} * \text{base} * \text{height}$
- Base = Length from p_1 to $p_2 = \text{sqrt}[(x_2-x_1)^2 + (y_2-y_1)^2]$
- Therefore, height =
 $|x_1y_2 + x_2y_3 + x_3y_1 - x_1y_3 - x_2y_1 - x_3y_2| / \text{sqrt}[(x_2-x_1)^2 + (y_2-y_1)^2]$
which is the distance from p_3 to line p_1p_2

Triangle

- If $p_1=(x_1,y_1)$, $p_2=(x_2,y_2)$, $p_3(x_3,y_3)$ then area of $\Delta p_1p_2p_3$
 $= |x_1y_2 + x_2y_3 + x_3y_1 - x_1y_3 - x_2y_1 - x_3y_2|/2$
- Sign of $(x_1y_2 + x_2y_3 + x_3y_1 - x_1y_3 - x_2y_1 - x_3y_2)$ is positive if and only if P_3 is to the left of the line P_1P_2 .
- So, we can use it to check whether a point, P_3 , lies to the left of the line, determined by two other points P_1P_2 , as well as find the distance.

Quickhull

CONVEXHULL (S, l, r)

Let P be the set containing points above (l, r)

Let Q be the set containing points below (l, r)

return {QUICKHULL (P, l, r) $\cup$ (l, r) $\cup$ QUICKHULL (Q, l, r)}

QUICKHULL (S, l, r)

//S is a set of points where rightmost lowest and leftmost highest
//points are r and l.

if S={ } **then return** ()

else if S={l, r} **then return** (l, r) // a single convex hull edge

else

z = index of a point that is furthest (max distance) from xy.

Let A be the set containing points strictly right of (x, z)

Let B be the set containing points strictly right of (z, y)

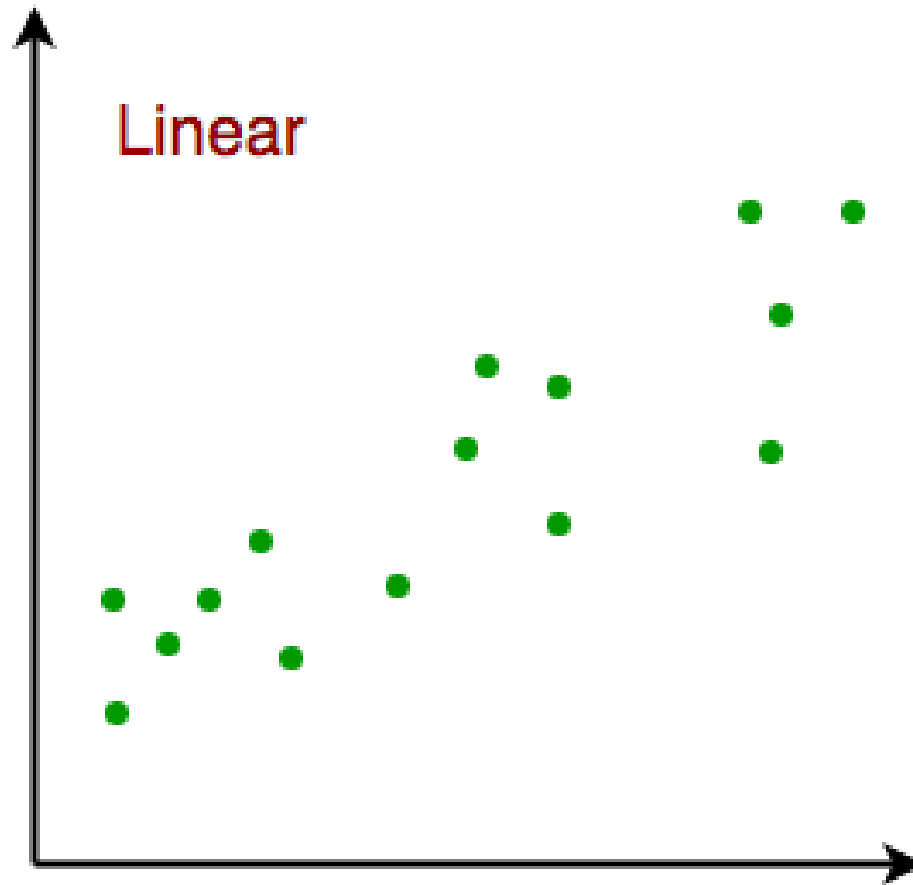
return {QUICKHULL (A, x, z) $\cup$ (z)

$\cup$ QUICKHULL (B, z, y)}

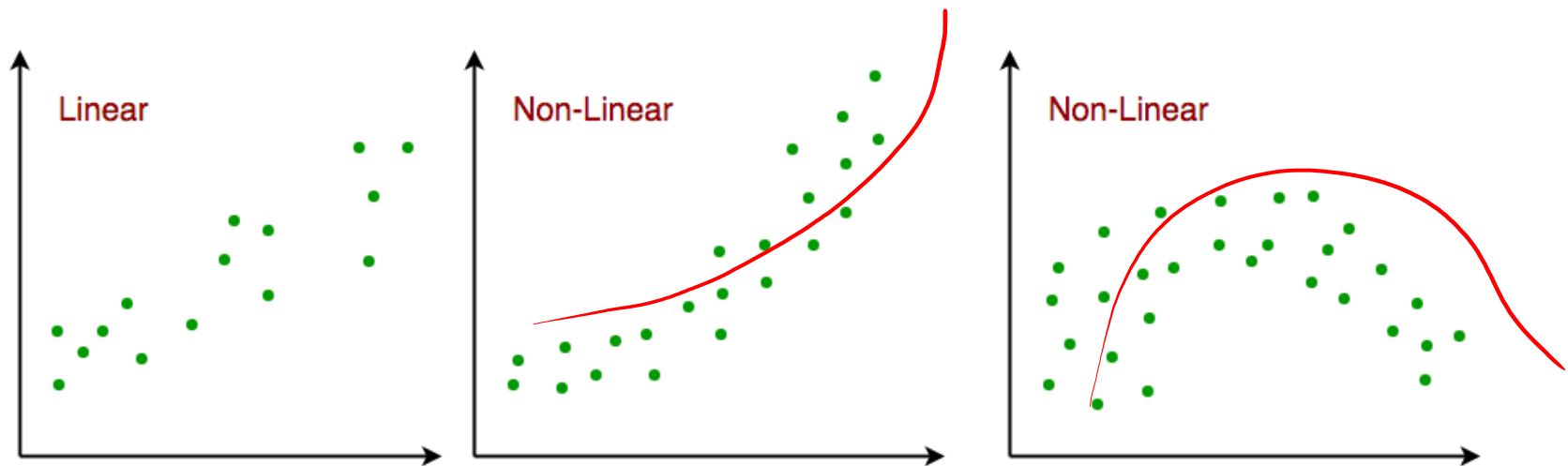
Efficiency of Quickhull Algorithm

- Finding point farthest away from line P_1P_2 can be done in linear time
- Time efficiency:
 - worst case: $\Theta(n^2)$ (as quicksort)
 - average case: $\Theta(n)$ (under reasonable assumptions about distribution of points given)
- If points are not initially sorted by x-coordinate value, this can be accomplished in $O(n \log n)$ time
- Several $O(n \log n)$ algorithms for convex hull are known

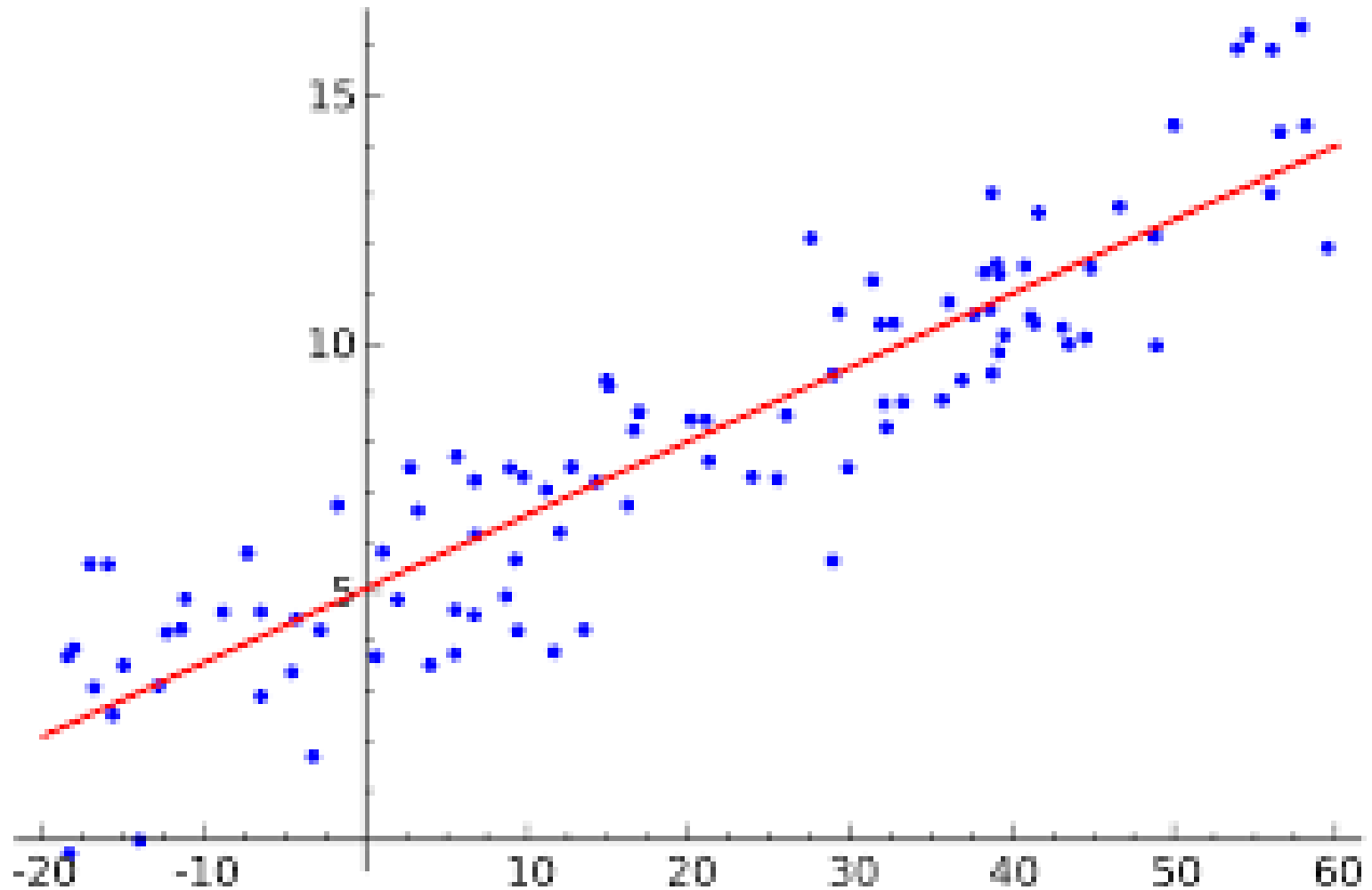
Machine Learning



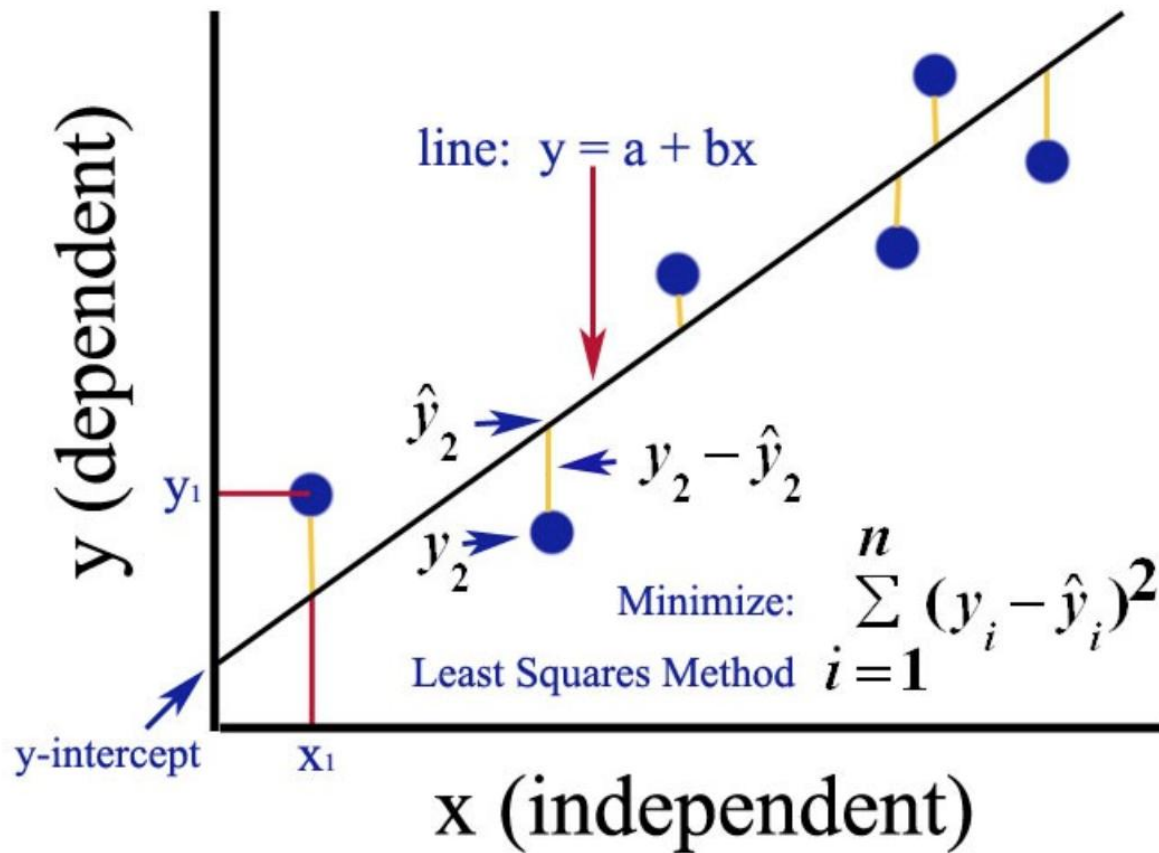
Machine Learning



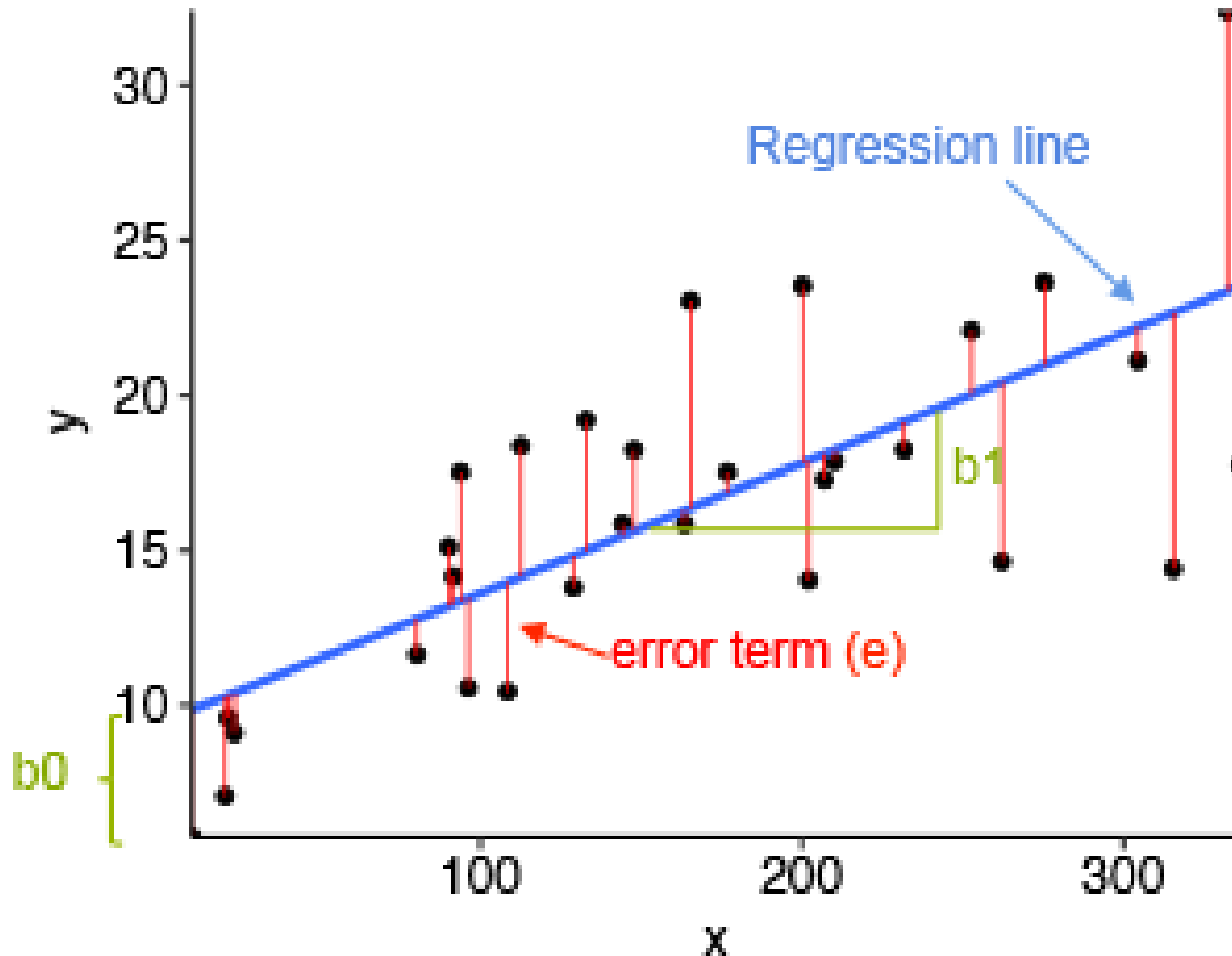
Linear Regression



Linear Equation



Reduce Error of Linear Regression



Quiz # : 15 minutes each question

Question 1 (10 points) :

Question 2

